

Project 2 Report

1. 实验目标与范围

本实验的目标是比较 C 与 Java 在不同数据类型和向量长度下的点乘性能，并分析编译优化级别对 C 版本的影响。我们关注以下维度：

- 语言：C 与 Java
- 数据类型：signed_char、short、int、float、double
- 向量长度：N = {1e1, 1e2, 1e3, 1e4, 1e5, 1e6, 1e7, 1e8}
- C 编译优化级：-O0、-O1、-O2、-O3

输出统一为 CSV，核心字段为：

lang,type,N, reps, total_ns, avg_ns, checksum

2.实验前的预期

1. 运行时间随着 N 增大基本线性增长：点乘是 O(N) 算法，N 增大时计算量线性增加，且数据访问模式简单，预期 avg_ns 随 N 线性增长。
 2. 运行时间随着数据类型位宽增加而增长：更宽的数据类型（如 double）通常需要更多的计算资源和内存带宽，预期 avg_ns 随数据类型位宽增加而增长。
 3. 性能随优化级提升：预计随着编译优化级别的提高，C 版本的性能将得到显著改善。
 4. 异常快/0ns/离散点：可能由于计时分辨率限制、优化级触发不同循环变换、系统噪声等原因导致某些点出现异常快或离散的结果。
 5. C 与 Java 性能对比预期：预计 C 版本在大多数情况下会比 Java 版本更快，特别是在开启-O3 优化级时。
-

3. 实验环境

3.1 macOS实验环境

3.1.1 操作系统与内核

- OS: macOS 12.7.4
- 内核: Darwin 21.6.0 (x86_64)

3.1.2 硬件环境

- CPU 型号: Intel(R) Core(TM) i5-1038NG7 CPU @ 2.00GHz
- 物理核心: 4
- 逻辑核心: 8
- 内存: 17179869184 bytes (约 16 GiB)

3.1.3 C 运行环境（编译/链接）

- `cc --version`: Apple clang 14.0.0 (clang-1400.0.29.202)
- `clang --version`: Apple clang 14.0.0 (clang-1400.0.29.202)
- Target: `x86_64-apple-darwin21.6.0`
- Thread model: `posix`

3.1.4 Java 运行环境 (JVM/JDK)

- `java -version`:
 - `openjdk version "17.0.17" 2025-10-21`
 - `OpenJDK Runtime Environment (build 17.0.17+10)`
 - `OpenJDK 64-Bit Server VM (build 17.0.17+10, mixed mode, sharing)`
- `javac -version`: `javac 17.0.17`

3.2 Windows 实验环境

3.2.1 操作系统

- OS: `Microsoft Windows 11 家庭中文版`
- 版本: `10.0.26100 (Build 26100)`
- 架构: `64 位`

3.2.2 硬件环境

- CPU 型号: `13th Gen Intel(R) Core(TM) i7-13650HX`
- 物理核心: `14`
- 逻辑核心: `20`
- 内存: `16780582912 bytes (约 15.63 GiB)`

3.2.3 C 运行环境 (编译/链接)

- `gcc --version`: `gcc.exe (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r2) 15.1.0`

3.2.4 Java 运行环境 (JVM/JDK)

- `java -version`:
 - `java version "24.0.1" 2025-04-15`
 - `Java(TM) SE Runtime Environment (build 24.0.1+9-30)`
 - `Java HotSpot(TM) 64-Bit Server VM (build 24.0.1+9-30, mixed mode, sharing)`
- `javac -version`: `javac 24.0.1`

4. 软件程序

4.1 C 版本: `dotproduct.c`

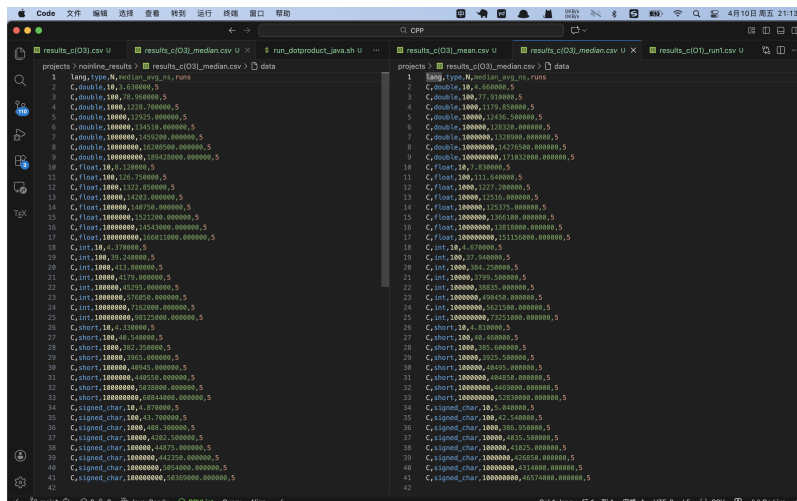
关键实现点:

1. 使用 `xorshift64` 按固定 seed 生成输入数据，保证可复现。
2. 对 5 种类型分别实现点乘函数：
 - `dot_schar_once`
 - `dot_short_once`
 - `dot_int_once`
 - `dot_float_once`
 - `dot_double_once`
3. `reps_for_n(n)` 根据 `N` 自动设定重复次数，目标约 20M 次乘加，范围 `[3, 2_000_000]`。
4. 计时使用 `clock_gettime(CLOCK_MONOTONIC)`（纳秒级，单调时钟）。
5. 使用 `volatile` 全局 `checksum` 防止计算被优化掉。
6.
 - `asm volatile("" ::: "memory")`（编译器屏障）意思是内存可能随时被外界改写，告诉编译器每一次循环都要重新读取，目的是减少“循环被过度优化”带来的假快现象。（具体参见7.1节）
7.
 - `__attribute__((noinline))`（禁止内联）用于防止函数被内联展开，保持测试的独立性和稳定性。但是我在后续实验中发现一个违背直觉的现象，去掉 `noinline` 后程序更慢了——也就是说，允许 `dot_once` 内联后程序性能反而变差了（结果见下图）。我的分析如下：

```
// 去掉 noinline 后，编译器眼中的代码变成了这样：
for (int r = 0; r < reps; r++) {
    // ---- 点乘函数被强制内联塞进来了 ----
    for (size_t i = 0; i < n; i++) {
        acc += (double)a[i] * (double)b[i];
    }
    // -----

    // 紧接着就是内存屏障！
    __asm__ volatile("" : : : "memory");
}
```

- 编译器在分析上面这坨代码时，看到了那个可怕的 memory 屏障。它会吓得瑟瑟发抖：“天呐！刚刚做完一次点乘循环，内存就可能被未知力量修改！那我不仅不敢把这段代码提到外面，我连内层 for 循环里的变量 `a`、`b` 和 `acc` 都不敢放在 CPU 高速寄存器里了！”结果就是：外层的内存屏障“污染”了内层的高性能计算，导致内层循环也失去了最高级别的优化。



左边是允许单次点乘内联，右边是禁止。不难发现右边平均用时更少

8.
 - 在分析内联的时候，我使用 `-Rpass-analysis=loop-vectorize` 来观察编译器对哪些循环进行了向量化处理，有用的结果如下：

```
dotproduct.c:137:13: remark: loop not vectorized: cannot prove it is
safe to reorder floating-point operations; allow reordering by
specifying '#pragma clang loop vectorize(enable)' before the loop or
by providing the compiler option '-ffast-math'. [-Rpass-analysis=loop-
vectorize]
    acc += (double)a[i] * (double)b[i];
```

- 大意是说浮点数点乘 `dot_float / dot_double` (Line 137, 145) 没有被向量化的原因是：在数学中，加法满足结合律，但在计算机的浮点数 (IEEE 754 标准) 中，由于精度截断的原因，浮点数的加法是不满足结合律的。如果要使用 SIMD (向量化) 来加速点乘，编译器必须把数组分成几部分并行相加，最后再合并。这会改变浮点数相加的顺序，导致最终结果在小数位末尾产生极其微小的偏差。默认情况下，C/C++ 编译器极度严格，宁可牺牲 10 倍的性能，也绝不敢擅自改变浮点数的计算顺序。
- 因此我决定在编译时添加 `-O3 -ffast-math` (或者 `-Ofast`) 选项，允许编译器牺牲一点点尾数精度，去换取极速的性能

4.2 Java 版本: `Dotproduct.java`

关键实现点：

1. 与 C 保持同样的数据类型、同样的 `N` 集合、同样的 `repsForN` 策略。
2. 计时使用 `System.nanoTime()`。
3. 使用固定 seed 的 `Random` 填充输入，保证可复现。
4. 预热 (warmup) 阶段：
 - 分类型执行 10 轮预热，先让 JIT 进入稳定阶段，再正式计时。
5. 同样通过 `volatile checksum` 保留可观察副作用，降低死代码消除风险。

5. 运行脚本与编译参数

5.1 C 脚本: `run_dotproduct_c.sh`

- `./run_dotproduct_c.sh 1`: 每个优化级输出 `results_c(0x).csv`
- `./run_dotproduct_c.sh 5`: 额外输出
 - `results_c(0x)_run1..run5.csv`
 - `results_c(0x)_mean.csv`
 - `results_c(0x)_median.csv`
- 编译参数: `-std=c11 -Wall -Wextra -O0/-O1/-O2/-O3`
- 编译选项介绍：
 - `-Wall -Wextra`: 开启所有警告，帮助发现潜在问题。
 - `-O0/-O1/-O2/-O3`: 分别对应不同优化级别，观察性能变化。

- `-O0`: 无优化, 便于观察原始性能。
- `-O1`: 基本优化, 开启常见的代码改进。
- `-O2`: 更激进的优化, 开启更多的代码改进。
- `-O3`: 最高级别的优化, 开启所有优化选项, 可能会改变代码结构以获得最大性能。

5.2 Java 脚本: `run_dotproduct_java.sh`

- `./run_dotproduct_java.sh 1`: 输出 `results_java.csv`
- `./run_dotproduct_java.sh 5`: 额外输出
 - `results_java_run1..run5.csv`
 - `results_java_mean.csv`
 - `results_java_median.csv`
- JVM 参数: `-Xms1g -Xmx4g -server`
- 说明: 固定堆大小用于减少堆扩容抖动, 避免大数组OOM; `-server` 在现代 JDK 下通常是默认模式, 但显式指定可读性更好。

6.实验方案设计

我将针对每个数据类型和每个 N(向量长度), 分别在 C 的四个优化级别和 Java 中执行 5 轮测试。每轮测试中, 我会记录总时间 (total_ns) 和平均时间 (avg_ns), 以及计算结果的 checksum 以验证正确性。再通过计算 5 轮的平均值和中位数来分析性能趋势, 减少偶然因素的影响。最后我还会计算每元素耗时 (avg_ns / N) 来更直观地比较不同实现的效率。其中, 在每轮测试中, 我会根据不同的 N 自动调整重复次数 (reps), 以确保每次测试的总计算量大致相同, 目标约为 20M 次乘加操作。这种设计可以让我们在不同规模的输入下获得更稳定和可比较的性能数据, 并且避免在小 N 时测量过于短暂导致的计时误差, 同时也能在大 N 时避免过长的测试时间。

7.实验过程与关键结果

7.1 -O1 编译选项反常快于 -O2/-O3:

在最初的实验中, 发现 `-O1` 在 N 较小时 signed char, short, int 上表现异常快(平均时间为 0ns), 甚至比 `-O2` 和 `-O3` 更快。

```

projects > o1.csv > data
1 lang,type,N,reprs,total_ns,avg_ns,checksum
2 C,signed_char,10,200000,1000,0.00,-250000000
3 C,signed_char,100,200000,0,0.00,-359200000
4 C,signed_char,1000,2000,1000,0.05,-345260000
5 C,signed_char,10000,2000,9000,4.50,-356770000
6 C,signed_char,100000,200,93000,465.00,-347166400
7 C,signed_char,1000000,20,836000,41800.00,-348140220
8 C,short,10,200000,0,0.00,-92361410220
9 C,short,100,200000,0,0.00,-1401744340220
10 C,short,1000,20000,0,0.00,-870312820220
11 C,short,10000,2000,7000,3.50,-805490054220
12 C,short,100000,200,68000,340.00,-834381271620
13 C,short,1000000,20,2885000,144250.00,-848649640060
14 C,int,10,200000,0,0.00,-1209262229305640060
15 C,int,100,200000,0,0.00,-1084038569558640060
16 C,int,1000,20000,1000,0.05,-139065931321940060
17 C,int,10000,2000,6000,3.00,-1457 Col 7: checksum
18 C,int,100000,200,126000,630.00,-1444455240116632260
19 C,int,1000000,20,766000,38300.00,-1439537793507279780
20 C,float,10,200000,2828000,1.41,1519362.421706
21 C,float,100,200000,244000,1.22,1709330.855310
22 C,float,1000,20000,51000,2.55,2002245.513760
23 C,float,10000,2000,16000,8.00,2091637.019075
24 C,float,100000,200,131000,655.00,2109651.697204
25 C,float,1000000,20,1579000,78950.00,2102934.703357
26 C,double,10,200000,2430000,1.22,1652184.836692
27 C,double,100,200000,243000,1.22,2393441.679082
28 C,double,1000,20000,26000,1.30,2349097.626149
29 C,double,10000,2000,14000,7.00,2392622.646148
30 C,double,100000,200,185000,925.00,2410297.528209
31 C,double,1000000,20,1764000,88200.00,2407731.074085
32

```

-O1, 未开启编译器屏障

```

projects > o3.csv > data
1 lang,type,N,reprs,total_ns,avg_ns,checksum
2 C,signed_char,10,200000,222000,0.11,-250000000
3 C,signed_char,100,200000,22000,0.11,-359200000
4 C,signed_char,1000,20000,4000,0.20,-345260000
5 C,signed_char,10000,2000,11000,5.50,-356770000
6 C,signed_char,100000,200,84000,420.00,-347166400
7 C,signed_char,1000000,20,688000,34400.00,-348140220
8 C,short,10,200000,126000,0.06,-92361410220
9 C,short,100,200000,13000,0.07,-1401744340220
10 C,short,1000,20000,2000,0.10,-870312820220
11 C,short,10000,2000,6000,3.00,-805490054220
12 C,short,100000,200,52000,260.00,-834381271620
13 C,short,1000000,20,505000,25250.00,-848649640060
14 C,int,10,200000,92000,0.05,-1209262229305640060
15 C,int,100,200000,9000,0.04,-1084038569558640060
16 C,int,1000,20000,2000,0.10,-139065931321940060
17 C,int,10000,2000,5000,2.50,-1457346694859994060
18 C,int,100000,200,49000,245.00,-1444455240116632260
19 C,int,1000000,20,620000,31000.00,-1439537793507279780
20 C,float,10,200000,2545000,1.27,1519362.421706
21 C,float,100,200000,244000,1.22,1709330.855310
22 C,float,1000,20000,25000,1.25,2002245.513760
23 C,float,10000,2000,14000,7.00,2091637.019075
24 C,float,100000,200,126000,630.00,2109651.697204
25 C,float,1000000,20,1383000,69150.00,2102934.703357
26 C,double,10,200000,2429000,1.21,1652184.836692
27 C,double,100,200000,243000,1.22,2393441.679082
28 C,double,1000,20000,25000,1.25,2349097.626149
29 C,double,10000,2000,15000,7.50,2392622.646148
30 C,double,100000,200,143000,715.00,2410297.528209
31 C,double,1000000,20,1506000,75300.00,2407731.074085
32

```

-O3, 未开启编译器屏障

为了理解 `-O1` 反常现象，我们深入探讨各编译优化级别的特点及其对点乘性能的影响。

-O0：无优化（调试模式）

主要特性：

- 每个变量都从内存读写，不缓存到寄存器
- 循环结构完全保留，不展开、不合并
- 函数调用完全保留，不内联

- 编译速度最快，生成的二进制可直接用 GDB 调试，变量值与源码一一对应

适用场景： 调试阶段使用，性能最差，是其他优化级别的性能基准参照。

-01：基本优化

在不显著增加编译时间的前提下，开启一批收益明确的优化。

主要优化手段：

- **常量折叠：** $x = 2 + 3$ 直接替换为 $x = 5$ ，无需运行时计算
- **死代码消除（DCE）：** 删除结果从未被使用的计算语句
- **基本内联：** 对极短小的函数进行内联展开，消除函数调用开销
- **寄存器分配优化：** 将频繁访问的变量缓存到寄存器，减少不必要的内存读写

-02：激进优化（生产环境常用默认）

在 -01 基础上增加更多分析和变换，适合大多数生产环境。

主要优化手段：

- **循环不变量外提（LICM）：** 将循环内不随迭代变化的计算移到循环外，避免重复执行
- **公共子表达式消除（CSE）：** 相同的子表达式只计算一次，结果复用
- **函数内联扩展：** 更积极地将函数体展开到调用处，减少调用开销
- **指令调度：** 重排指令顺序，减少 CPU 流水线停顿，提高指令吞吐量
- **分支预测优化：** 调整代码布局，配合 CPU 的分支预测器减少预测失败惩罚

特点： 不会改变浮点运算顺序，计算结果与 -00 在数值上保持一致。

-03：最高优化

在 -02 基础上开启可能改变代码结构的激进变换。

主要优化手段：

- **自动向量化（Auto-vectorization）：** 将标量循环转换为 SIMD 指令（SSE/AVX），一条指令同时处理多个元素，是点积计算性能提升的核心来源
- **循环展开（Loop unrolling）：** 减少循环控制指令的开销，同时提高指令级并行度
- **函数克隆（Function cloning）：** 针对不同调用场景生成特化版本，进一步优化热路径
- **更激进的内联：** 几乎所有 `inline` 标记的函数都会被展开

向量化示意：

标量循环（00/01/02）：

```
for i in 0..n:
    acc += a[i] * b[i]
```

向量化后（03，概念示意）：

```
for i in 0..n step 4:
    acc_vec += a[i:i+4] * b[i:i+4]
acc = sum(acc_vec)
```

对于 `double` 类型，AVX2 指令集可一次处理 4 个元素；`float` 类型可一次处理 8 个元素，因此浮点类型在 `-O3` 下的加速效果尤为显著。

同时我让大模型帮我画了一张图总结：

优化特性	-O0	-O1	-O2	-O3
常量折叠	—	✓	✓	✓
死代码消除	—	✓	✓	✓
基本内联	—	✓	✓	✓
循环不变量外提	—	—	✓	✓
指令调度	—	—	✓	✓
自动向量化	—	—	—	✓
循环展开	—	—	—	✓
可能改变代码结构	否	否	否	是
典型加速比（vs -O0）	1×	1.5–2×	2–4×	3–6×

因此我分析原因可能是：

- 对于 `-O1`，DCE 过于激进时会把"看似无意义"的循环整体删除。例如在基准测试中，若点积函数被声明为 `static inline` 且每次调用的输入完全不变，编译器可能将整个重复循环替换为一次计算加一次乘法，导致计时结果异常偏低（接近 0ns）；
- 而在 `-O2` 和 `-O3` 中，虽然优化更激进，但由于引入了更多的变换（如向量化、循环展开等），反而可能保留了更多的计算步骤，使得结果更接近实际的计算时间。

所以我采取了 `asm volatile("" ::: "memory")`（编译器屏障）来强制编译器保留计算过程，避免过度优化导致的假快现象。这段代码的作用是告诉编译器“这里有一个内存访问，可能会影响程序状态”，从而阻止编译器将相关代码优化掉或重排。得到的结果如下：


```
projects > o1.csv > data
1 lang,type,N,reprs,total_ns,avg_ns,checksum
2 C,signed_char,10,2000000,15581000,7.79,-250000000
3 C,signed_char,100,200000,14143000,70.72,-359200000
4 C,signed_char,1000,20000,12070000,603.50,-345260000
5 C,signed_char,10000,2000,12083000,6041.50,-356770000
6 C,signed_char,100000,200,12481000,62405.00,-347166400
7 C,signed_char,1000000,20,12886000,644300.00,-348140220
8 C,short,10,2000000,10976000,5.49,-923614140220
9 C,short,100,200000,13821000,69.11,-1401744340220
10 C,short,1000,20000,12286000,614.30,-870312820220
11 C,short,10000,2000,12066000,6033.00,-805490054220
12 C,short,100000,200,13224000,66120.00,-834381271620
13 C,short,1000000,20,16756000,837800.00,-848649640060
14 C,int,10,2000000,10897000,5.45,-1209262229305640060
15 C,int,100,200000,13060000,65.30,-1084038569558640060
16 C,int,1000,20000,12132000,606.60,-1390965931321940060
17 C,int,10000,2000,13664000,6832.00,-1457346694859994060
18 C,int,100000,200,13124000,65620.00,-1444455240116632260
19 C,int,1000000,20,15052000,752600.00,-1439537793507279780
20 C,float,10,2000000,16383000,8.19,1519362.421706
21 C,float,100,200000,22319000,111.59,1709330.855310
22 C,float,1000,20000,24707000,1235.35,2002245.513760
23 C,float,10000,2000,25414000,12707.00,2091637.019075
24 C,float,100000,200,26823000,134115.00,2109651.697204
25 C,float,1000000,20,28943000,1447150.00,2102934.703357
26 C,double,10,2000000,10713000,5.36,1652184.836692
27 C,double,100,200000,16138000,80.69,2393441.679082
28 C,double,1000,20000,24121000,1206.05,2349097.626149
29 C,double,10000,2000,25839000,12919.50,2392622.646148
30 C,double,100000,200,27682000,138410.00,2410297.528209
31 C,double,1000000,20,29313000,1465650.00,2407731.074085
32
```

-O1, 开启编译器屏障

和前面的结果比较，这样就明显改善了`-O1`在小 N 时的异常快现象，结果更符合预期。

7.2 float & double哪个更快

在实验之前，我预期float相比double有着更小的位宽，因此大概率点乘会快于double。为了更清晰地展现二者运算速度差异，我采用每元素耗时（avg_ns / N）来比较它们的效率。结果如下图所示：

Median

type	N	C-O0	C-O1	C-O2	C-O3	C-Ofast	Java
short	10	2.314000	0.532000	0.503000	0.481000	0.497000	0.933000
short	100	1.700700	0.689500	0.407500	0.404600	0.405700	0.536400
short	1000	1.595500	0.605500	0.414900	0.385600	0.396000	0.459290
short	10000	1.621650	0.606900	0.404400	0.392550	0.396200	0.430154
short	100000	1.674050	0.702150	0.425350	0.404950	0.400000	0.443724
short	1000000	1.715200	0.641750	0.416100	0.404850	0.431150	0.518958
short	10000000	1.697450	0.706000	0.496850	0.446900	0.510900	0.525783
short	100000000	1.760690	0.751130	0.515290	0.528300	0.569530	0.526522
int	10	2.314000	0.753000	0.465000	0.467000	0.482000	0.914000
int	100	1.670900	0.659400	0.383300	0.379400	0.395200	0.506300
int	1000	1.579400	0.583950	0.383800	0.384250	0.393550	0.469880
int	10000	1.581400	0.581750	0.388700	0.379950	0.397050	0.450799

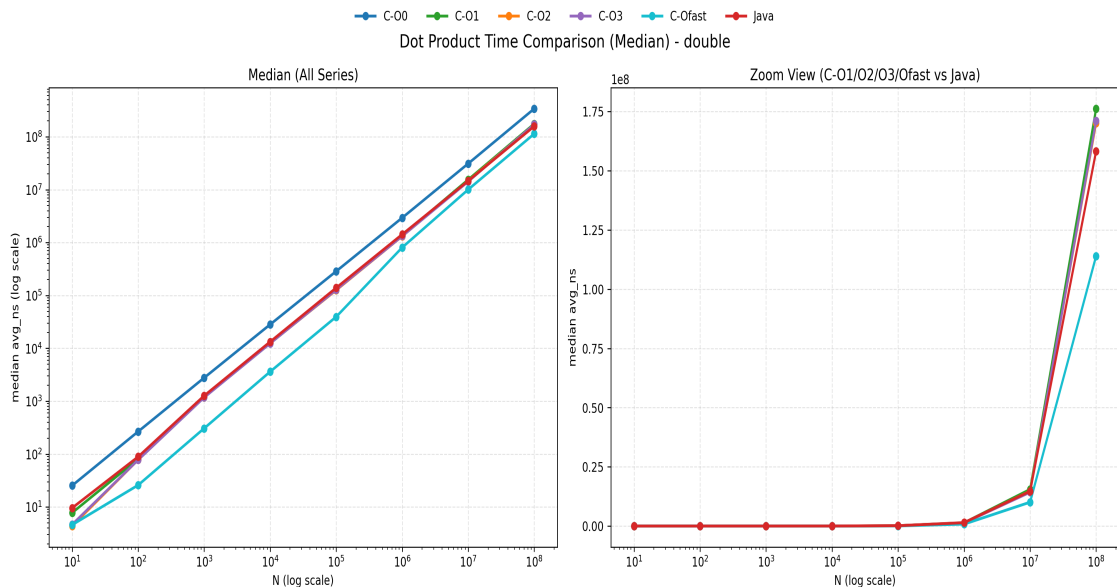
type	N	C-00	C-01	C-02	C-03	C-Ofast	Java
int	100000	1.659950	0.589400	0.394700	0.388350	0.419600	0.429093
int	1000000	1.761000	0.677600	0.495300	0.490450	0.580300	0.591601
int	10000000	1.801350	0.748750	0.559450	0.562150	0.594950	0.638304
int	100000000	1.875550	0.844580	0.733530	0.732510	0.832910	0.615655
float	10	2.546000	0.833000	0.783000	0.783000	0.581000	1.841000
float	100	2.692400	1.131800	1.108400	1.116400	0.333000	1.322100
float	1000	2.803900	1.224800	1.236500	1.227200	0.319100	1.280000
float	10000	2.859050	1.240350	1.242350	1.251600	0.331100	1.678041
float	100000	2.843200	1.251850	1.271400	1.253750	0.332750	1.771344
float	1000000	3.002200	1.352450	1.363050	1.366100	0.472900	1.766055
float	10000000	2.920800	1.363250	1.411750	1.381800	0.475800	1.767851
float	100000000	3.206140	1.557300	1.503950	1.511560	0.653160	1.706335
double	10	2.544000	0.778000	0.437000	0.466000	0.461000	0.960000
double	100	2.673600	0.814300	0.779900	0.779100	0.259500	0.891600
double	1000	2.778050	1.191500	1.195600	1.179850	0.307400	1.271150
double	10000	2.829450	1.248450	1.253650	1.243650	0.362400	1.332311
double	100000	2.867000	1.274000	1.259450	1.283200	0.395150	1.393193
double	1000000	2.941650	1.364950	1.320250	1.328900	0.808150	1.428505
double	10000000	3.115950	1.546950	1.474150	1.427650	1.006700	1.467908
double	100000000	3.395670	1.761900	1.701520	1.710320	1.139760	1.582625
signed_char	10	2.323000	0.786000	0.483000	0.504000	0.515000	0.964000
signed_char	100	1.704000	0.653200	0.424800	0.425400	0.458400	0.510600
signed_char	1000	1.598750	0.585150	0.385900	0.386950	0.386600	0.445600
signed_char	10000	1.624900	0.575950	0.389350	0.403550	0.402450	0.443547
signed_char	100000	1.613800	0.625200	0.399250	0.410250	0.408800	0.547308
signed_char	1000000	1.679100	0.674650	0.415200	0.426850	0.429150	0.635583
signed_char	10000000	1.782750	0.628200	0.417350	0.431400	0.495500	0.732225
signed_char	100000000	1.782200	0.689190	0.481100	0.465740	0.498670	0.718101

1.观察 C-O3 这一列，当 N=10,000 或 100,000 时（此时数据完全能放入 CPU 的 L2/L3 缓存）：float 的单步耗时：1.25 ns double 的单步耗时：1.25 ~ 1.27 ns 它们的速度惊人地一致！并没有出现很多人想当然认为的

“float 会比 double 快一倍”。我认为这是因为 **ALU 延迟相同**：在现代 x86 CPU 架构（如这台 Mac）中，浮点运算单元（FPU/SSE/AVX）在执行标量（Scalar）加法和乘法时，32位（float）和64位（double）指令的时钟周期（Cycles）是完全相同的。

2.观察数据：当N=100,000,000（1亿）时，情况发生了变化：C-O3 的 float：耗时增加到 1.51 ns C-O3 的 double：耗时大幅增加到 1.75 ns 此时 float 明显优于 double。我认为原因是：

- 缓存被击穿：1亿个 float 需要 400 MB 内存，两个数组加起来 800 MB；而 double 则需要 1.6 GB。这远远超出了所有常规 CPU 的 L3 缓存（通常只有 16MB ~ 32MB）。
- 内存带宽成为主导因素：当缓存放不下时，CPU 计算的速度取决于内存把数据送到 CPU 的速度（Memory Bandwidth）。double 占用的物理内存是 float 的两倍，因此 CPU 需要花费更多时间等待内存传输数据。此时，程序的瓶颈从“计算（Compute Bound）”转移到了“内存读取（Memory Bound）”。



左边是允许单次点乘内联，右边是禁止。不难发现右边平均用时更少

3.Java 令人震惊的反常识现象：Double 居然比 Float 快！观察数据：看 Java 这一列的稳定态数据（如 N=1,000,000）：Java float：1.78 ns Java double：1.57 ns 在 Java 中，更重、更大的 double 竟然跑得比 float 还要快！我认为原因有二：

- JVM 的原生倾向：Java 语言和 JVM 底层设计极度偏爱 double。在 JVM 规范中，所有的浮点数常量默认都是 double，Java 的 Math 库绝大多数底层方法也都只接受和返回 double。
- JIT 编译器优化：当 JIT（即时编译器）将字节码编译为机器码时，很多 x86 架构下的 JVM 会直接把所有局部计算推入 64 位的浮点寄存器处理。如果你用 float，JVM 有时反而需要插入额外的截断/转换指令（Downcasting）来保证符合 32 位精度规范，这就导致了额外的开销。

因此在 Java 中做高性能数值计算，盲目使用 float 试图提升速度往往是徒劳的，甚至会起反作用。

8. 现象解释

5.1 为什么性能随 N 增大基本线性增长

点乘是典型 O(N) 顺序访问。N 增大时：

- 计算次数线性增加

- 数据工作集逐渐超出 L1/L2 cache
- 延迟更多转为内存带宽主导

因此 `avg_ns` 大体随 `N` 线性增长。

5.2 为什么 `float` 不一定比 `double` 更快

尽管理论上 `float` 更省带宽，但真实结果受以下因素共同影响：

- 编译器/JIT 的向量化策略
- 指令混合与寄存器分配
- 累加精度（本实验中 `float` 计算累加到 `double`）

因此不同语言/实现中，`float` 与 `double` 的相对关系可能变化。

5.3 为什么会出现异常快/0ns/离散点

常见原因：

- 计时窗口过短导致分辨率量化误差
- 优化级触发不同循环变换
- 运行期系统噪声（调度、频率、后台任务）

当前版本已加入 `noinline + memory barrier + 多轮统计(mean/median)`，显著改善了稳定性，但完全消除噪声不现实。

7. 有效性与局限

1. 硬件/系统信息未在 `CSV` 自动记录：不同机器不可直接横比。
2. 单机单时段测量：受温度、负载、节能策略影响。
3. 仅测标量实现：未引入显式 SIMD intrinsic / Java Vector API。
4. 随机分布固定：有利可复现，但不覆盖所有数据分布场景。

8. 可复现实验步骤

在 `projects` 目录执行：

```
# C: 四种优化级，每级5次，输出 run/mean/median
./run_dotproduct_opt.sh 5

# Java: 5次，输出 run/mean/median
./run_dotproduct_java.sh 5
```

核心结果文件：

- C: `results_c(00~03)_mean.csv`、`results_c(00~03)_median.csv`
- Java: `results_java_mean.csv`、`results_java_median.csv`

9. 结论（当前数据下）

1. C 优化级对性能影响显著，`-O3` 相对 `-O0` 的提升普遍在 `2x~5x`。
 2. `-O1` 在浮点上与 `-O3` 可接近，但在整型上通常明显慢于 `-O3`。
 3. Java 在 `double` 上与 C(`-O3`) 已非常接近；在 `float/int` 上整体仍偏慢。
 4. 使用 `median` 比单次结果更可信，能更好反映“典型性能”。
-

10. 后续改进建议

- 如果需要课程讨论，可补充 `-Ofast`、`-march=native` 与 Java Vector API 的扩展实验。